Computer Organization and Architecture
(Code: CS256)

Lecture-21: Cache Mapping

Course Faculty

Dr. B. R. Bhowmik

Assistant Professor

Dept. of CSE

Website: https://sites.google.com/view/drbrbhowmik
https://sites.google.com/view/cs256-spring2020

Course Email: sanjanyatha@gmail.com

Elements of Cache Design

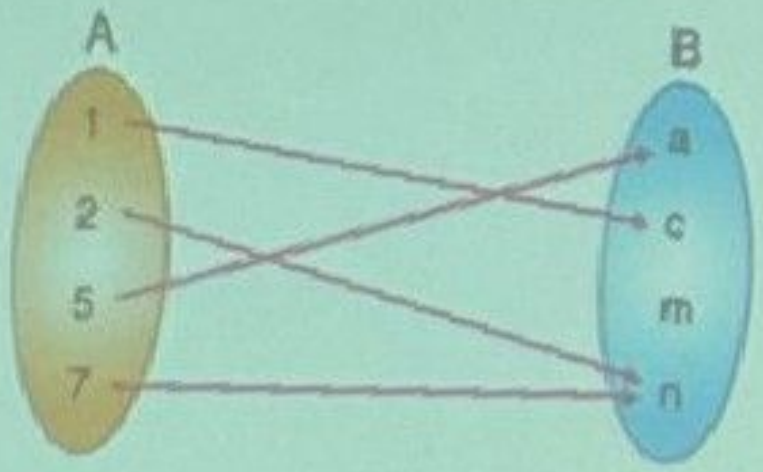
- Size
  - Cache Size,
  - Block Size
  - Line Size
- Addressing
  - Logical
  - Physical
- Mapping Function
  - Direct,
  - Associative,
  - Set Associative
- Write Policy
  - Write through,
  - Write back,
  - Write once
- Number of Caches
  - Single or Multi level,
  - Unified or Split
- Replacement Algorithms
  - Least recently used (LRU),
  - First in first out (FIFO),
  - Least frequently used (LFU),
  - Random

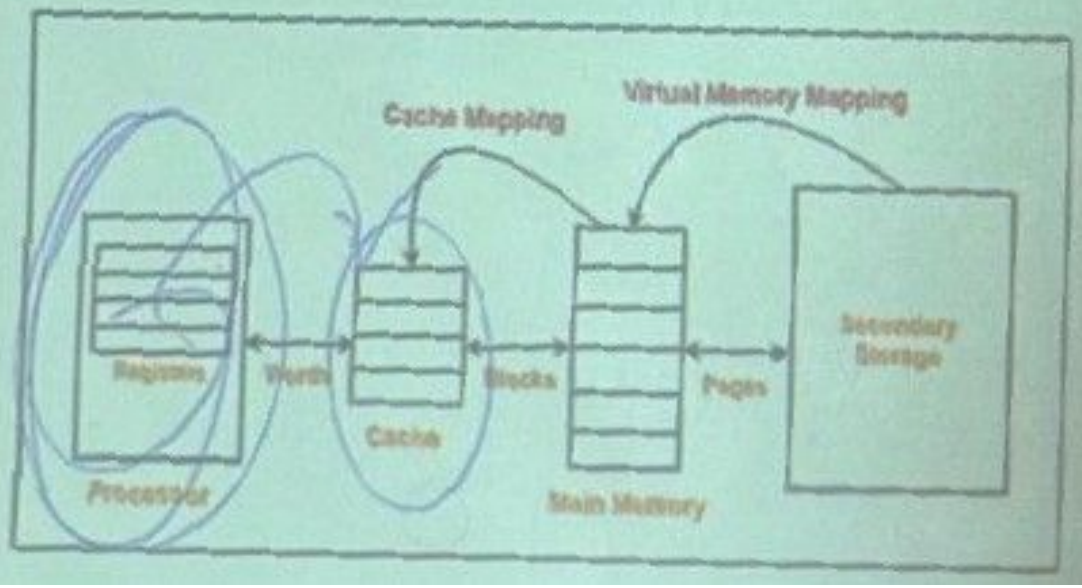
Cache Mapping

- Direct
- Associative
- Set Associative

Mapping Functions

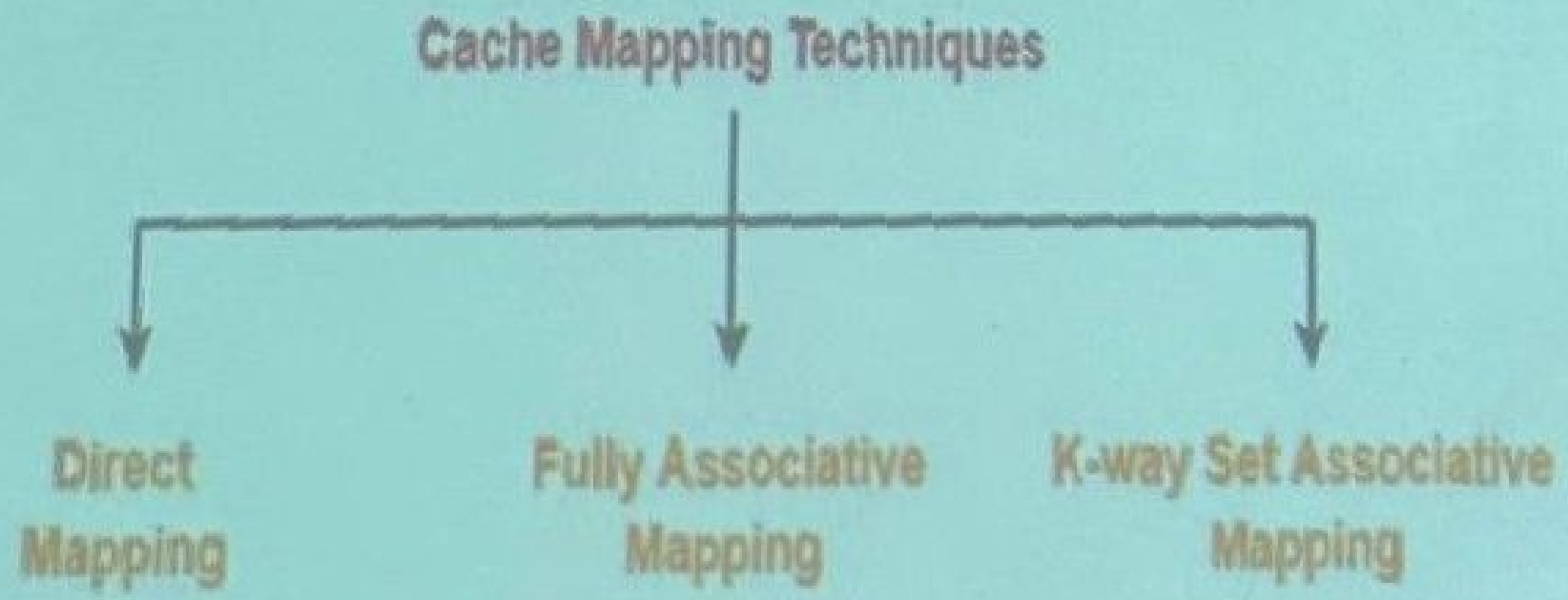
- A mapping function is the method used to locate a memory address within a cache.
- It is used when copying a block from main memory to the cache and it is used again when trying to retrieve data from the cache.



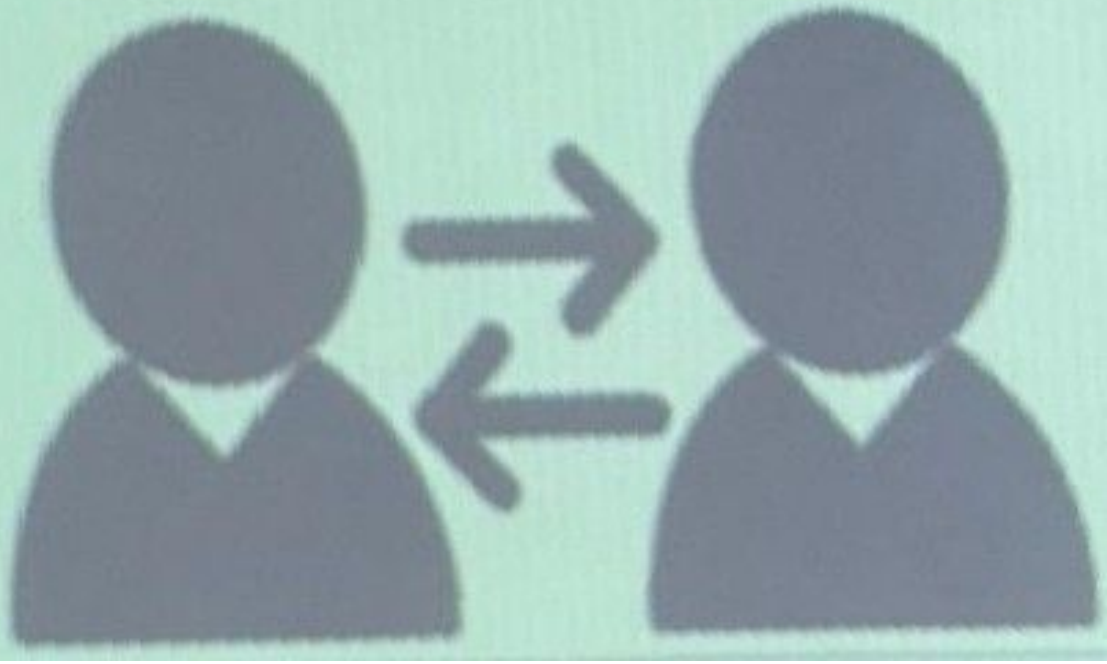


Mapping Types

- There are three kinds of mapping functions
  - Direct
  - *(Fully)* Associative
  - *(K-Way)* Set Associative

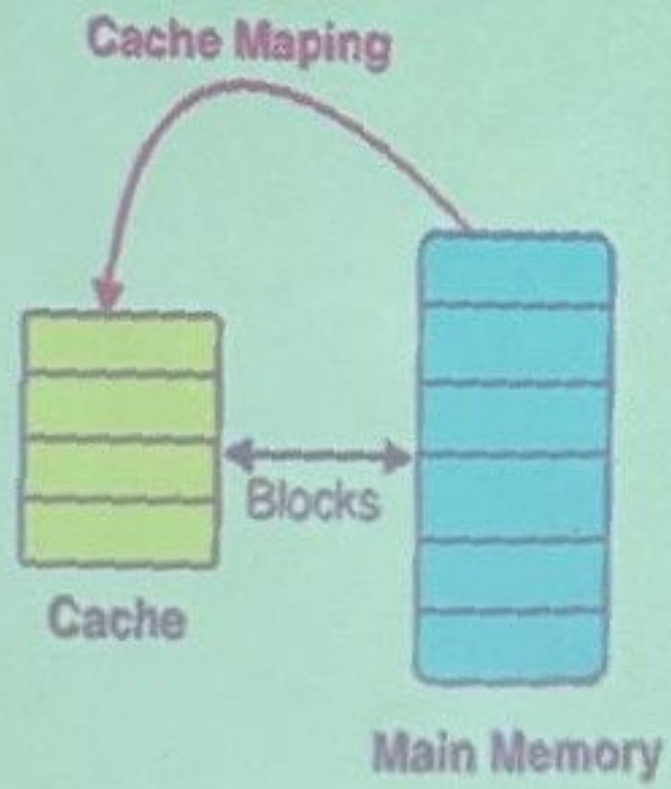


Direct Cache Mapping



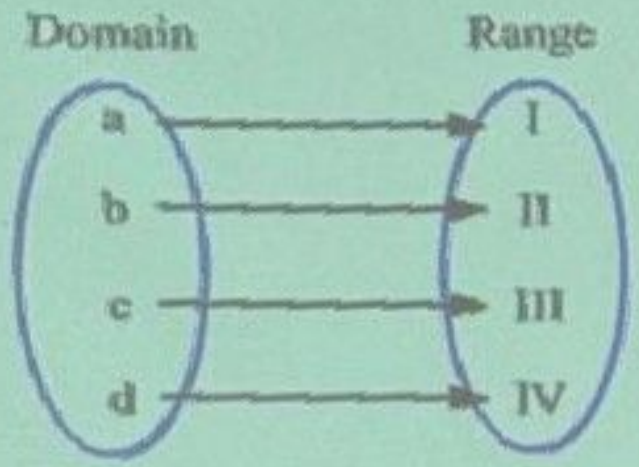
Cache/Memory Example

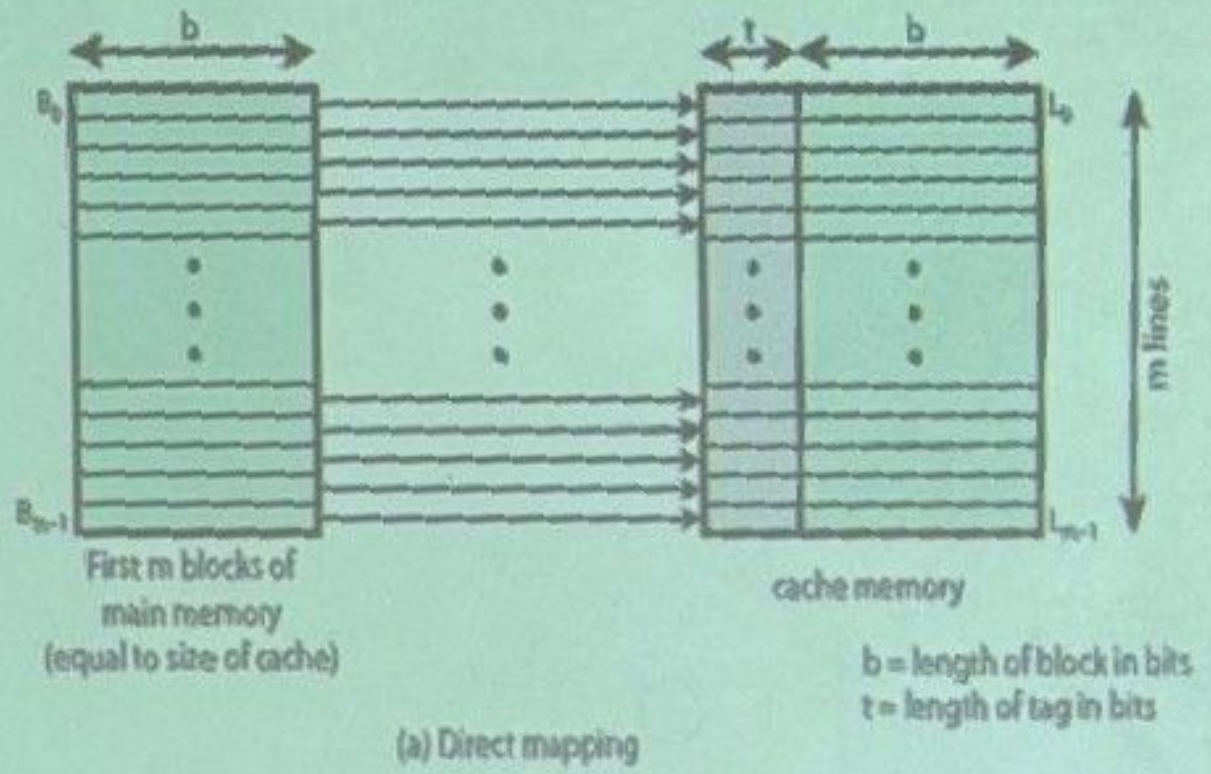
- Cache Size: 64 kByte
- Block size: 4 bytes.
- The cache has 16k (2^{14}) lines of 4 bytes
- Main memory: 16M bytes
- Address bus: 24-bit
- Blocks: 4M



Direct Mapping

- Each block of main memory maps to only one cache line.
  - if a block is in cache, it will always be found in the same place





(a) Direct mapping

Direct Mapping

- Line number is calculated using the following function

$$i = j \text{ modulo } m$$

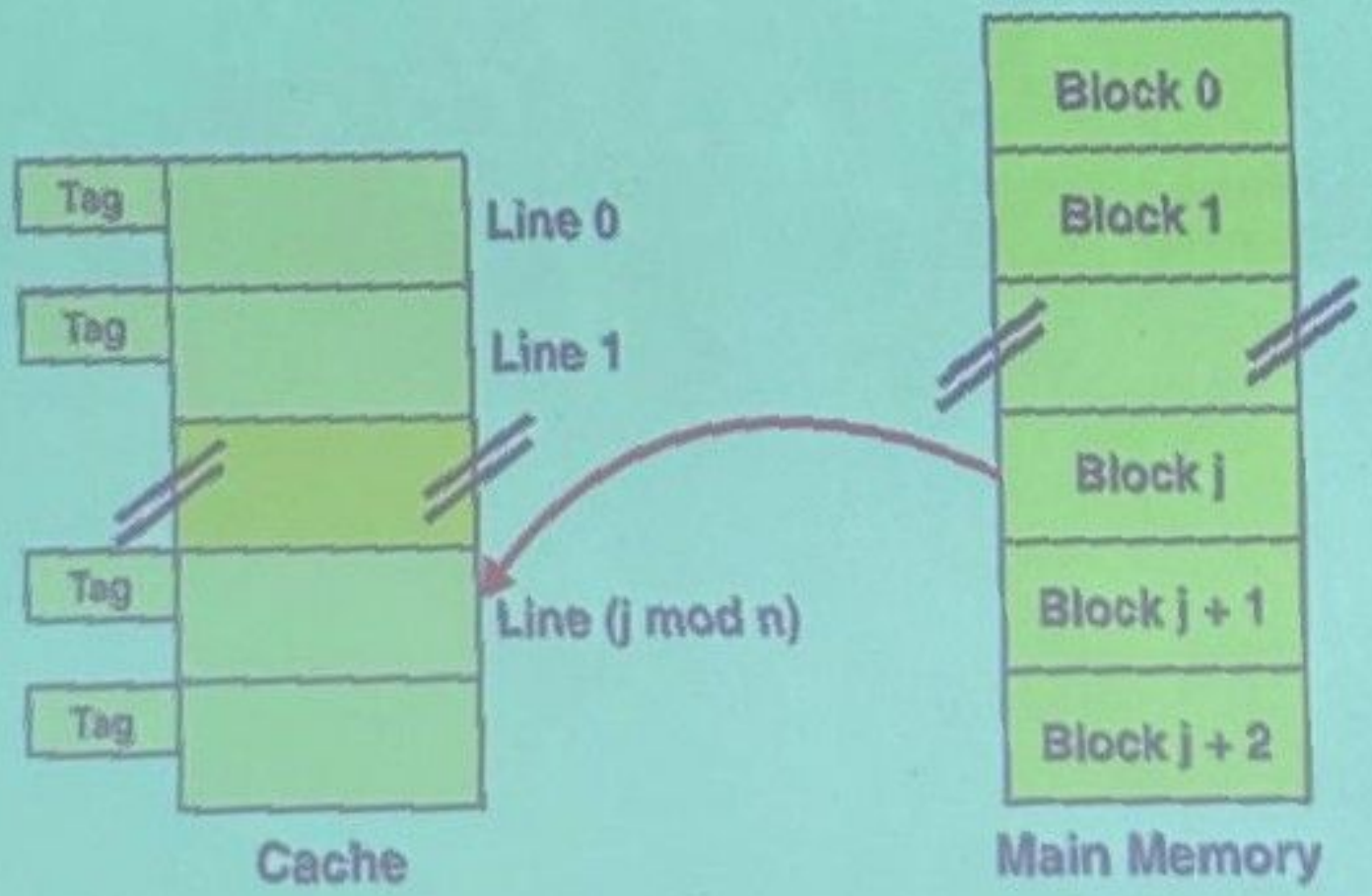
where

i = cache line number

j = main memory block number

m = number of lines in the cache

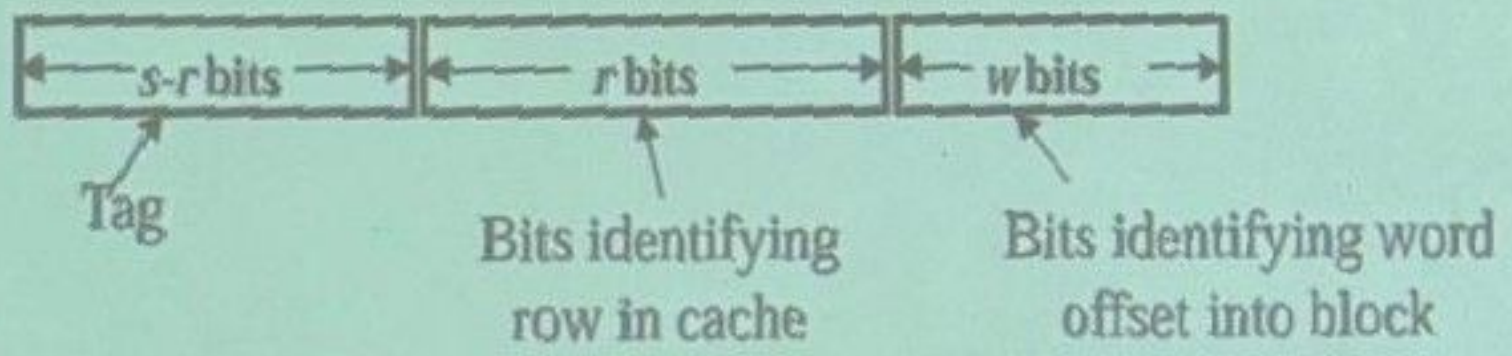
Direct Mapping



Direct Mapping

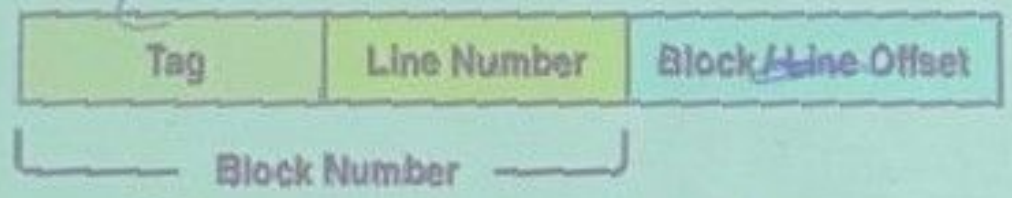
Direct Mapping Address Structure

- *Address is in two parts: Block Number and Block Offset*
- Least Significant **w** bits identify unique word within a block → ***Block Offset***
- Remaining bits (s) specify which block in memory → ***Block Number***
- *The s is divided into two fields*
  - Least significant r bits of these s bits identifies which line in the cache.
  - Most significant s-r bits uniquely identifies the block within a line of the cache
  - *Thus, each main memory address can by divided into three fields*



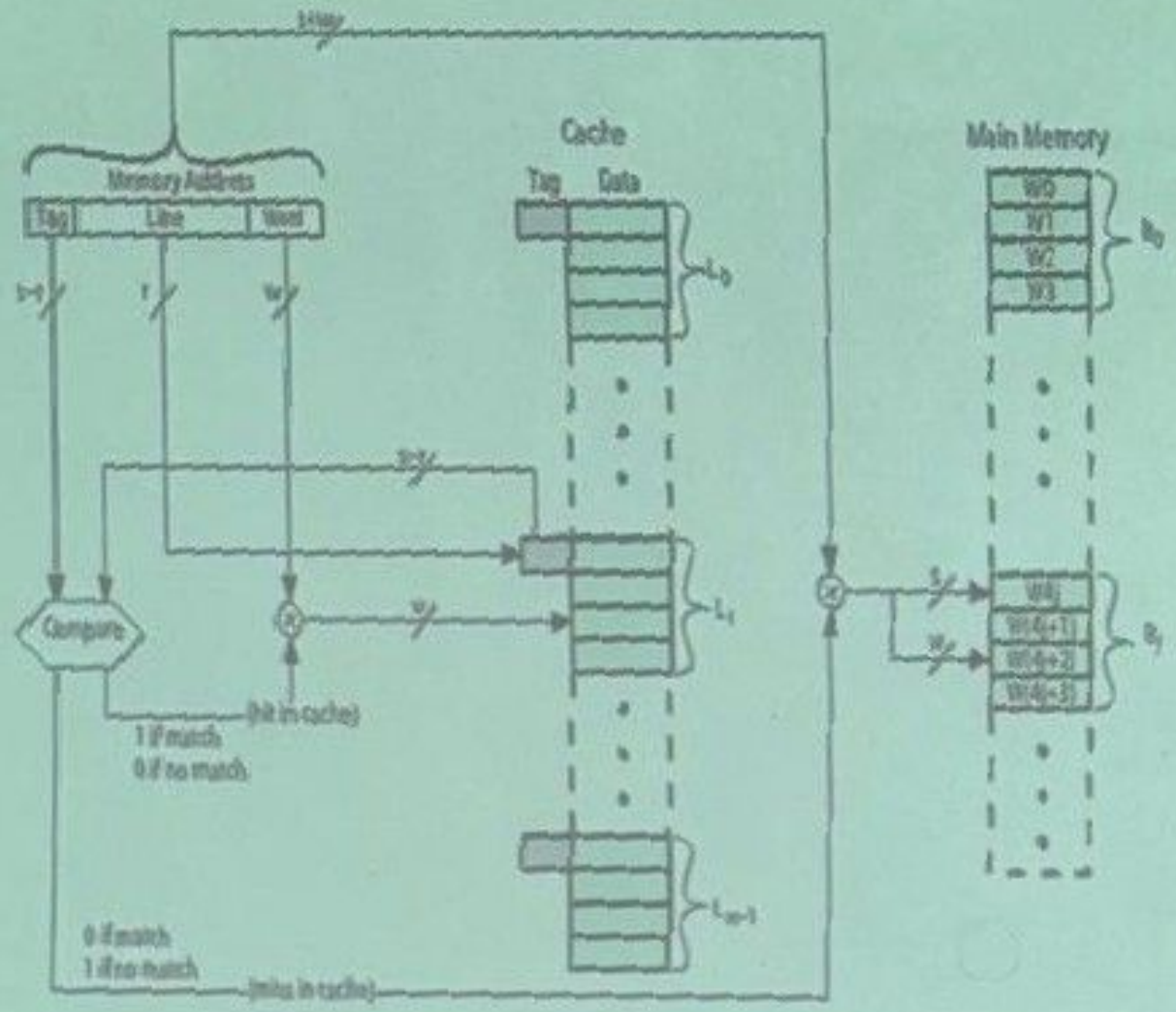
Direct Mapping Address Structure

- Why are the r-bits used to identify which line in cache?
- More likely to have unique r bits than s-r bits based on principle of **locality of reference**



Division of Physical Address in Direct Mapping

Direct Mapping Cache Organization



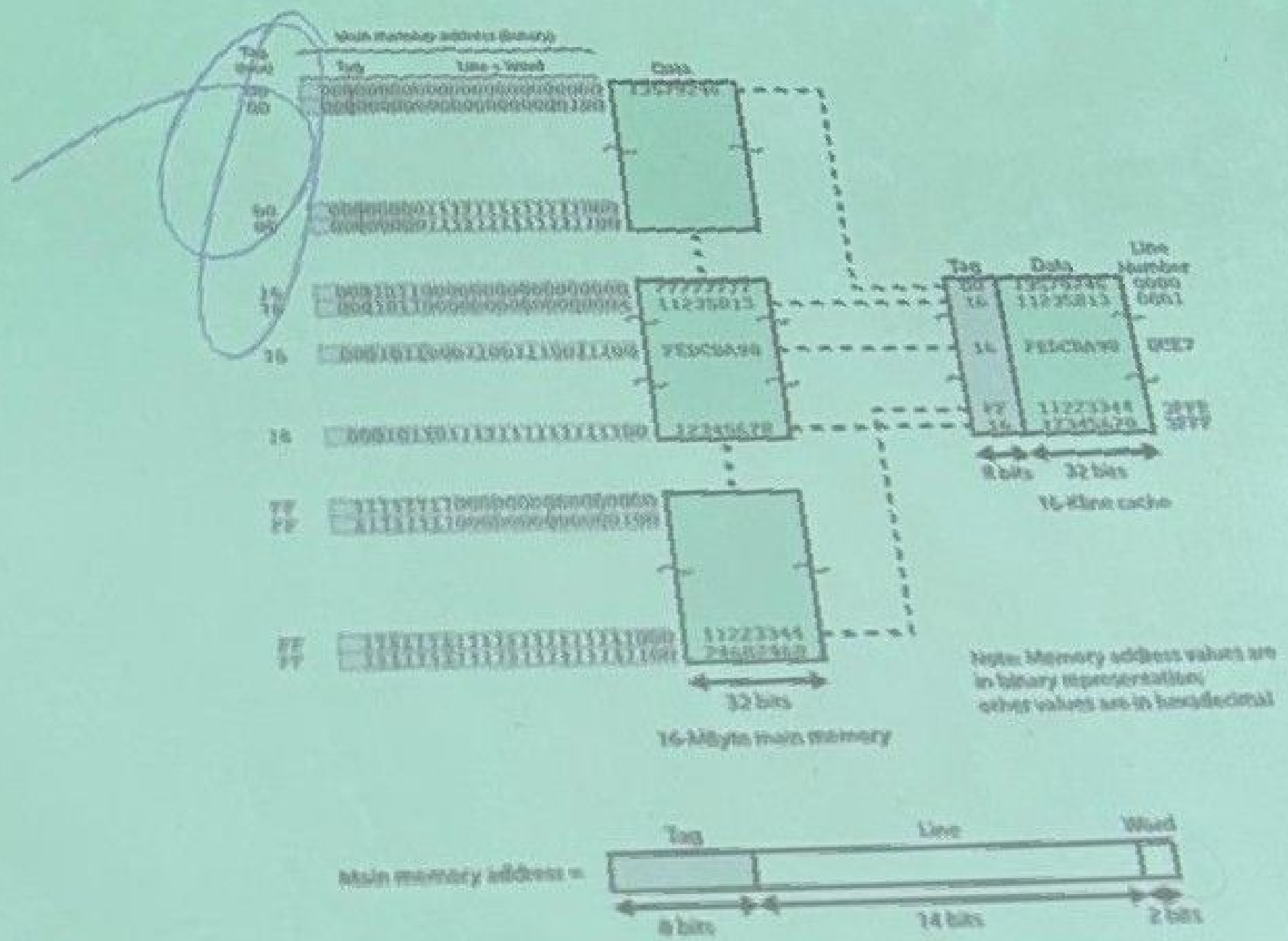
Direct Mapping Address Structure Example

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
- 8 bit tag (=22–14)
- 14 bit slot or line

Tag s-r	Line or slot r	Word w
8	14	2

- No two blocks in the same line have the same tag
- Check contents of cache by finding line and comparing tag

Direct Mapping Example



Direct Mapping Example 3

- Assume that a portion of the tags in the cache in our example looks like the table below. Which of the following addresses are contained in the cache?

a.) $438EE8_{16}$ b.) $F18EFF_{16}$ c.) $6B8EF3_{16}$ d.) $AD8EF3_{16}$

Tag (binary)	Line number (binary)	Addresses w/ block			
		00	01	10	11
0101 0011	1000 1110 1110 10				
1110 1101	1000 1110 1110 11				
1010 1101	1000 1110 1111 00				
0110 1011	1000 1110 1111 01				
1011 0101	1000 1110 1111 10				
1111 0001	1000 1110 1111 11				

Direct Mapping Pros & Cons

- **Pros:**
  - The direct mapping technique is simplex.
  - It is inexpensive to implement.
  - Fixed location for given block.

Direct Mapping Pros & Cons

❑ **Cons:**

- Its main disadvantage is that there is a fixed cache location for any given block.
- Thus, if a program happens to reference words repeatedly from two different blocks that map into the same line, then the blocks will be continually swapped in the cache, and the hit ratio will be low (a phenomenon known as ***thrashing***).

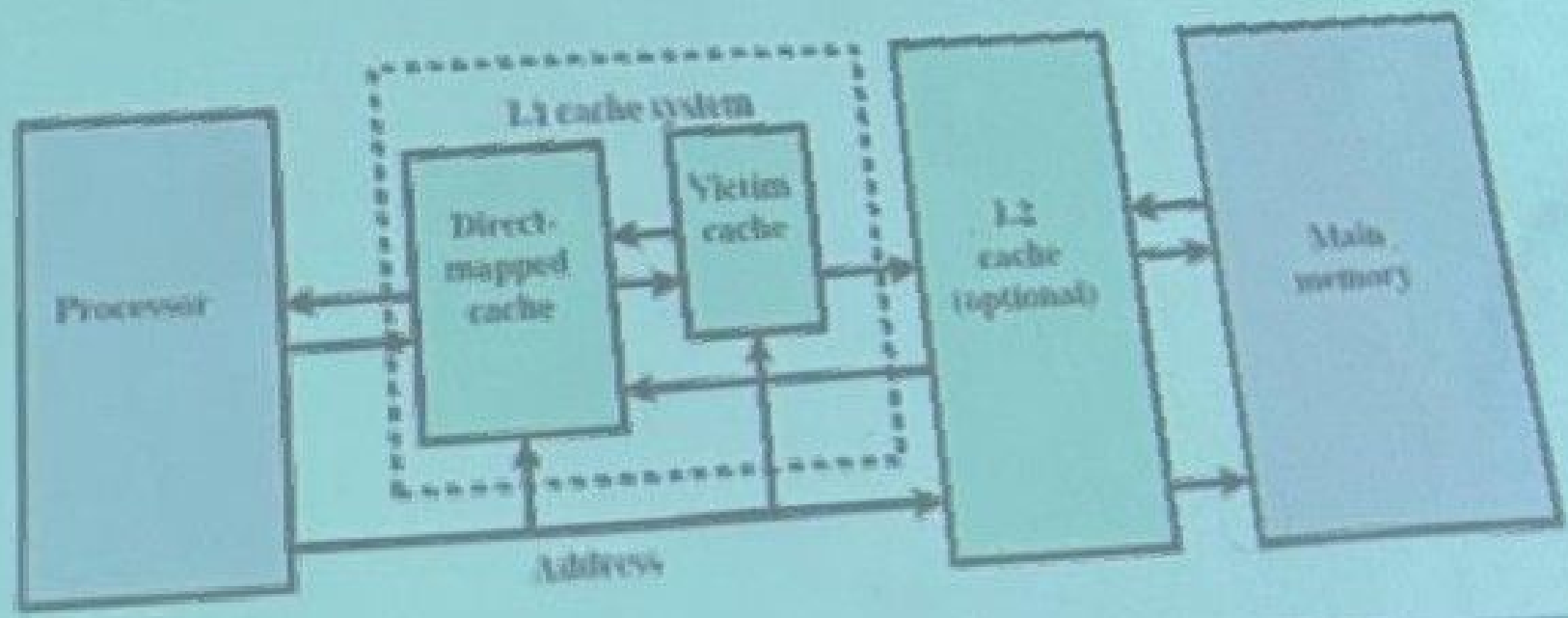
Direct Mapping Limitation

Cache line	Main Memory blocks held
0	0, m, 2m, 3m…2s-m
1	1,m+1, 2m+1…2s-m+1
…	
m-1	m-1, 2m-1,3m-1…2s-1

Solution: *Victim Cache*

Victim Cache

- *Victim cache is proposed as an approach to reduce the conflict misses of direct mapped caches without affecting its fast access time.*
- ***A small fully-associative back up cache, called victim cache, is added to a direct-mapped L1 cache to hold recently evicted cache lines.***

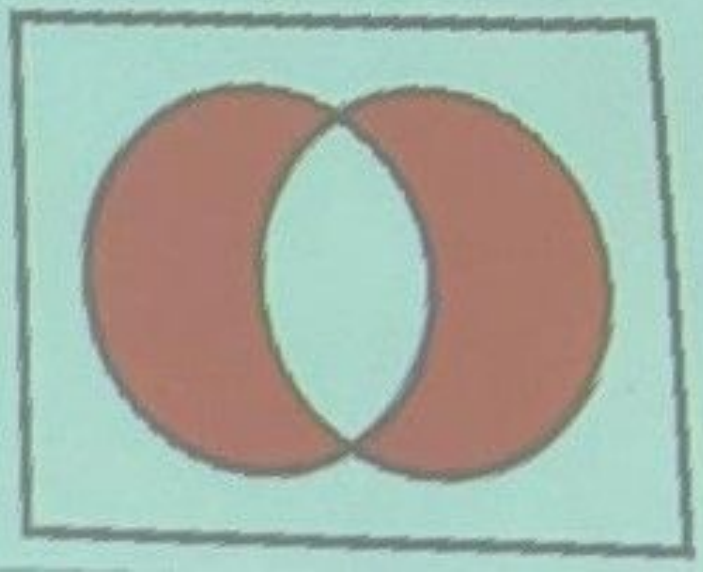


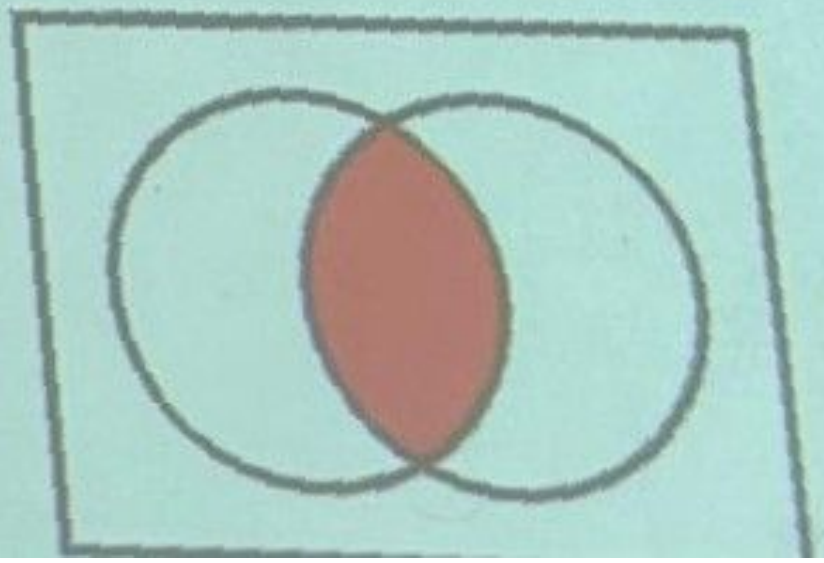
Victim Cache

- The L1 data cache is checked. If it holds the data requested, the data is returned.
- If the data is not in the L1 cache, the victim cache is checked.
- If it holds the data requested, the data is moved into the L1 cache and sent back to the processor. The data evicted from the L1 cache is put in the victim cache, and put at the end of the FIFO replacement queue.
- If neither of the caches holds the data, it is retrieved from memory, and put in the L1 cache.
- If the L1 cache needs to evict old data to make space for the new data, the old data is put in the victim cache and placed at the end of the FIFO replacement queue.
- Any data that needs to be evicted from the victim cache to make space is written back to memory or discarded, if unmodified.

Victim Cache

- The two caches (L1 and Victim) are **exclusive**.
- That means that the same data cannot be stored in both L1 and victim caches at the same time.





Victim Cache: Advantages

- Lowers miss penalty
- Remembers what was discarded
  - Already fetched
  - Use again with little penalty
- Is Fully associative
- Has size 4 to 16 cache lines
- Resides between direct mapped L1 cache and next memory level.
  - Less access time

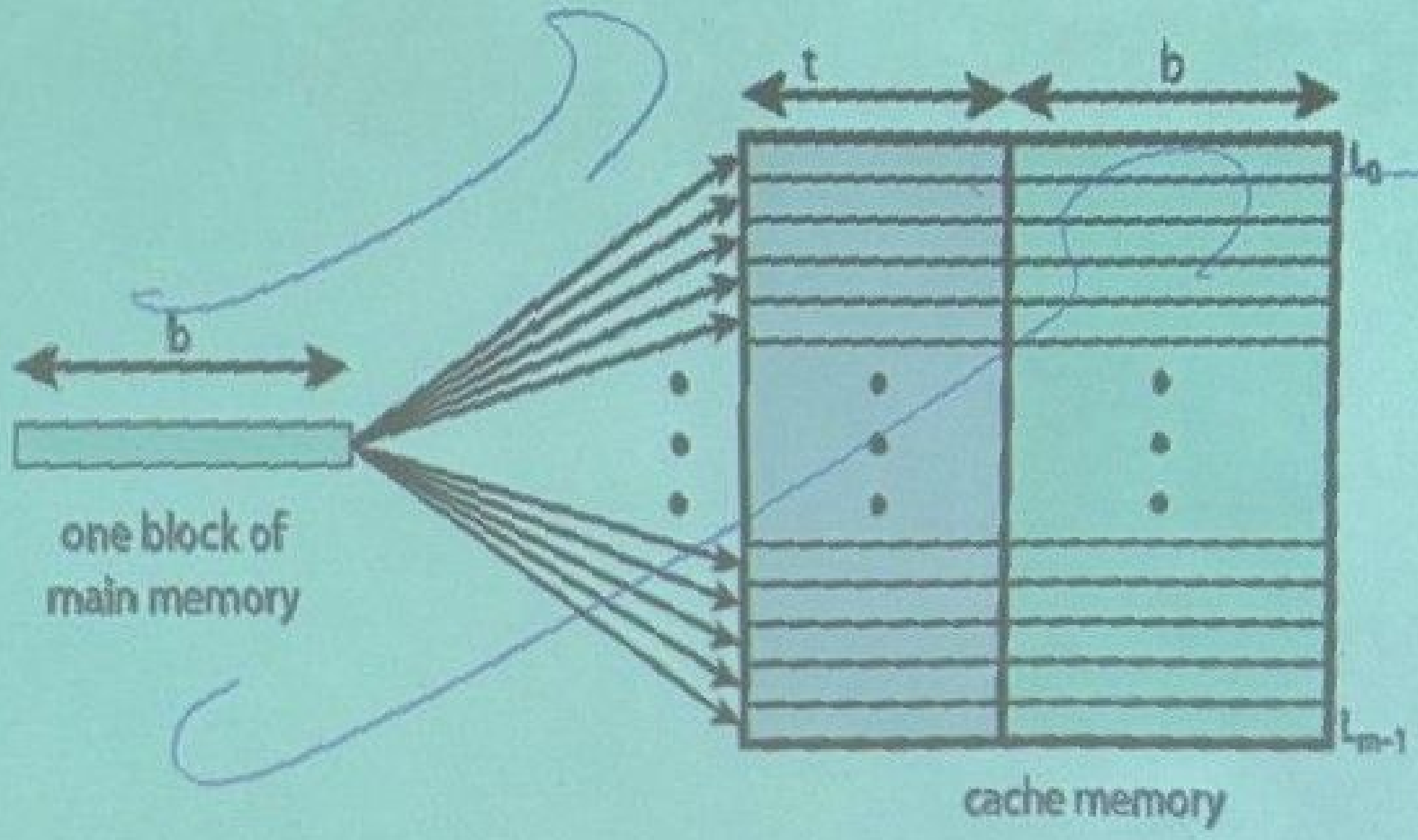
Fully Associative

Cache Mapping

Fully Associative Mapping

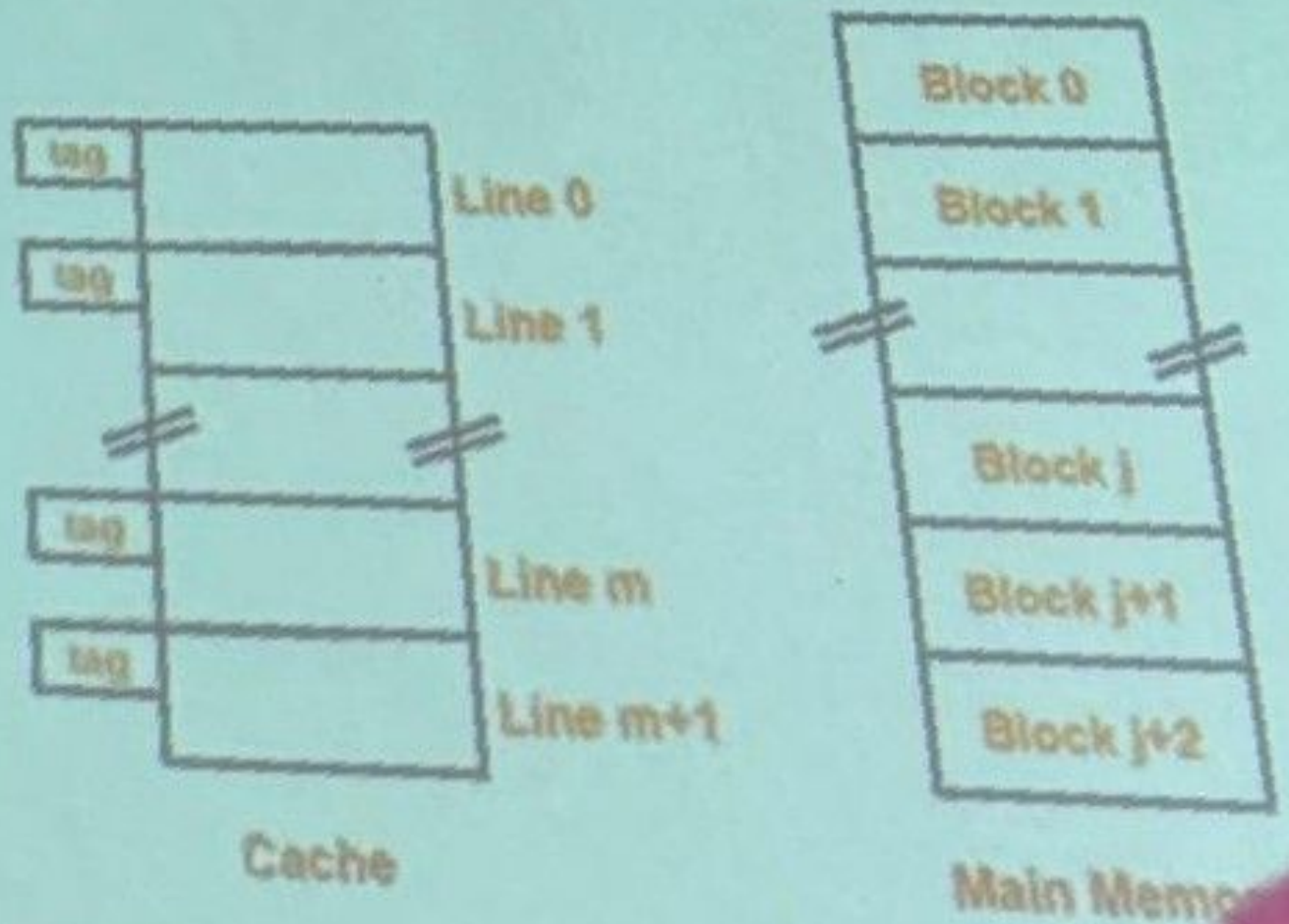
- Associative mapping overcomes the disadvantage of direct mapping.
- A main memory block can load into any line of cache.
- A fully associative cache permits data to be stored in any cache block, instead of forcing each memory address into one particular block.

Associative Mapping from Cache to Main Memory



Fully Associative Mapping

When data is fetched from memory, it can be placed in any unused block (line) of the cache. This way never have a conflict between two or more memory addresses which map to a single cache block.

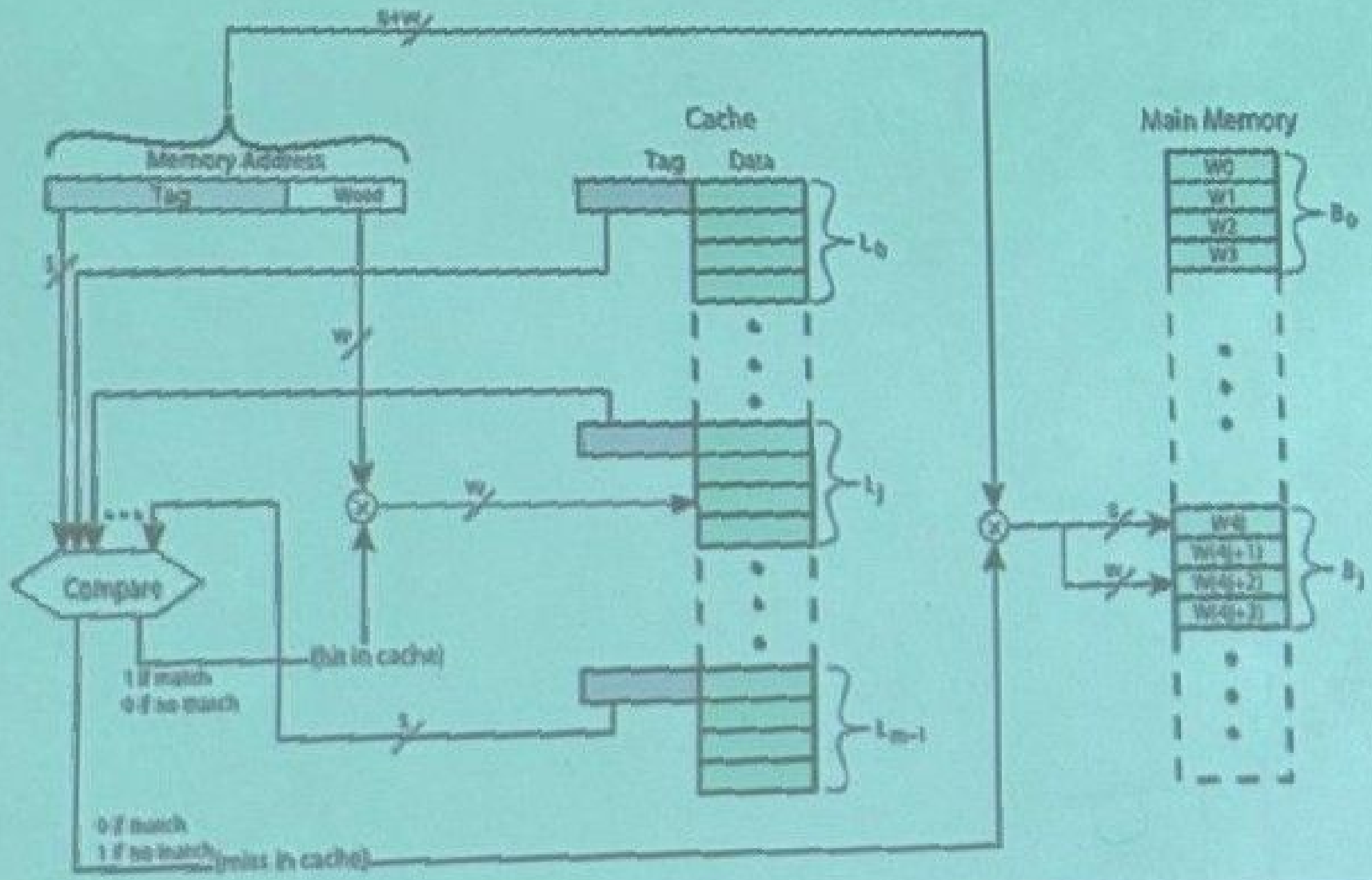


Fully Associative Mapping

- *A fully associative cache contains a single set with C ways, where C is the number of cache lines or blocks.*
- A fully associative cache is another name for a C-way set associative cache with one set.
- Synonymously called "*Associative Mapping*".
- Every line's tag must be examined for a match.

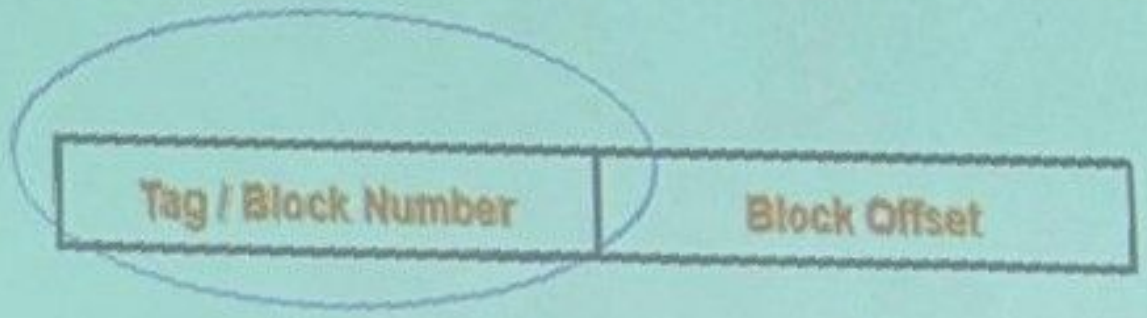
Fully Associative Cache Organization

- Every line's tag must be examined for a match.

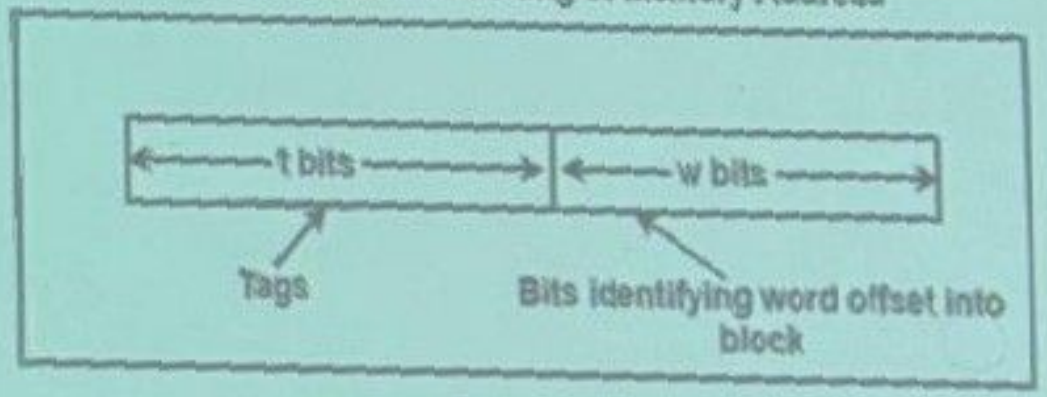


Fully Associative Mapping Address Structure

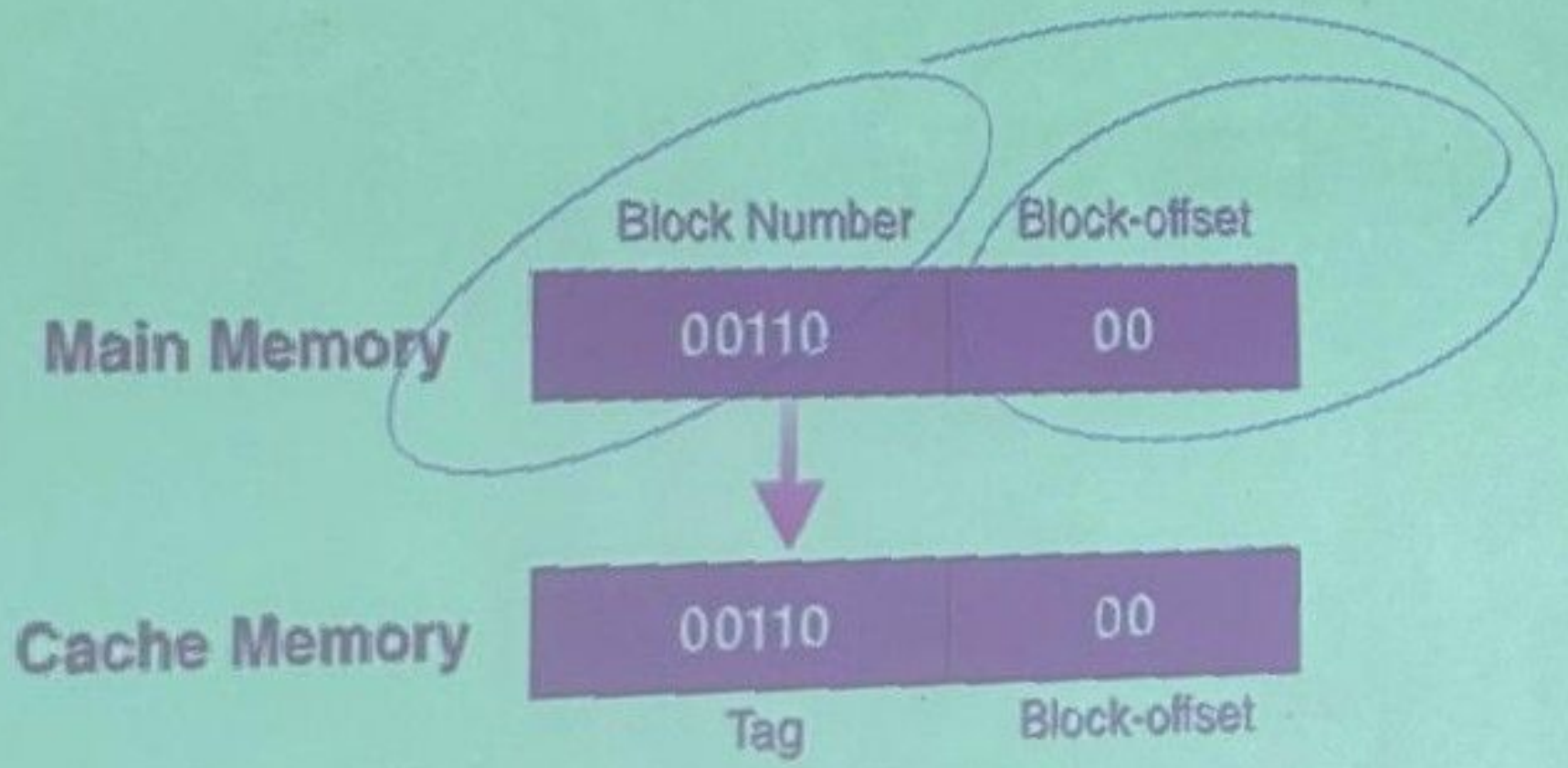
- Memory address is interpreted as:
  - Least significant **w** bits = **word** position within block
  - Most significant **s or t** bits = **tag** used to identify which block is stored in a particular line of cache



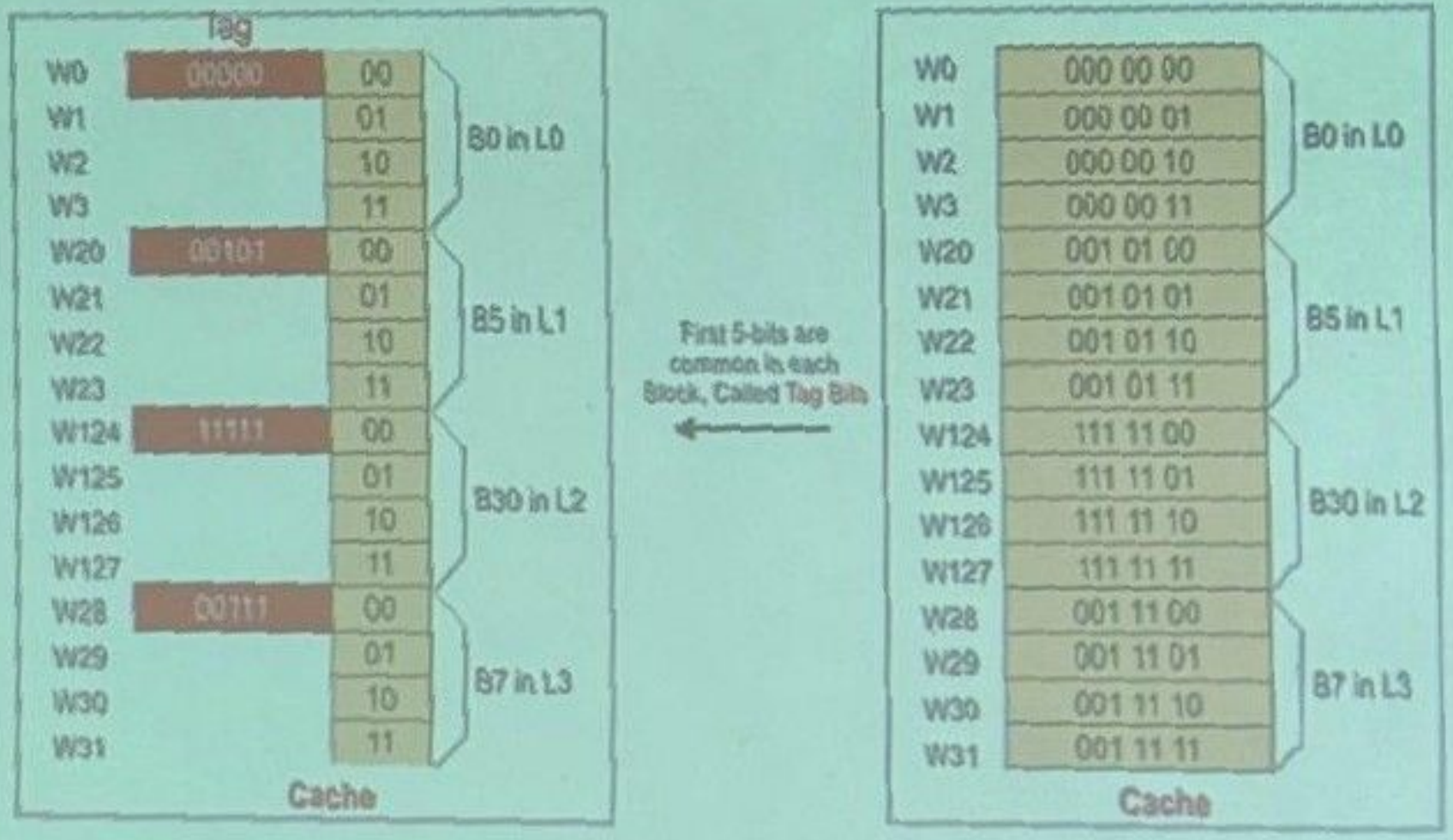
Associative Partitioning of Memory Address



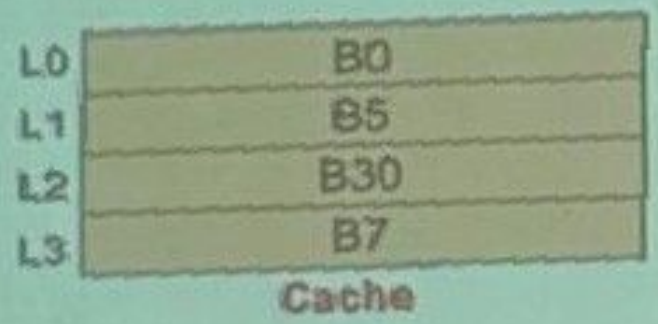
Fully Associative Mapping
Address Structure



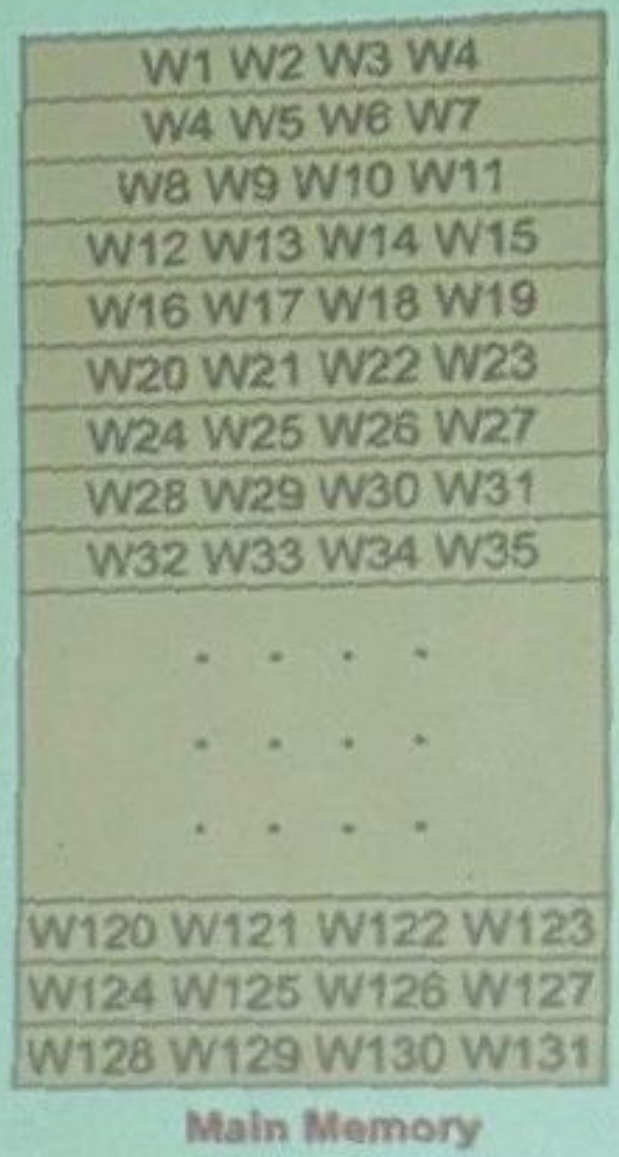
Fully Associative Mapping



Fully Associative Mapping Example

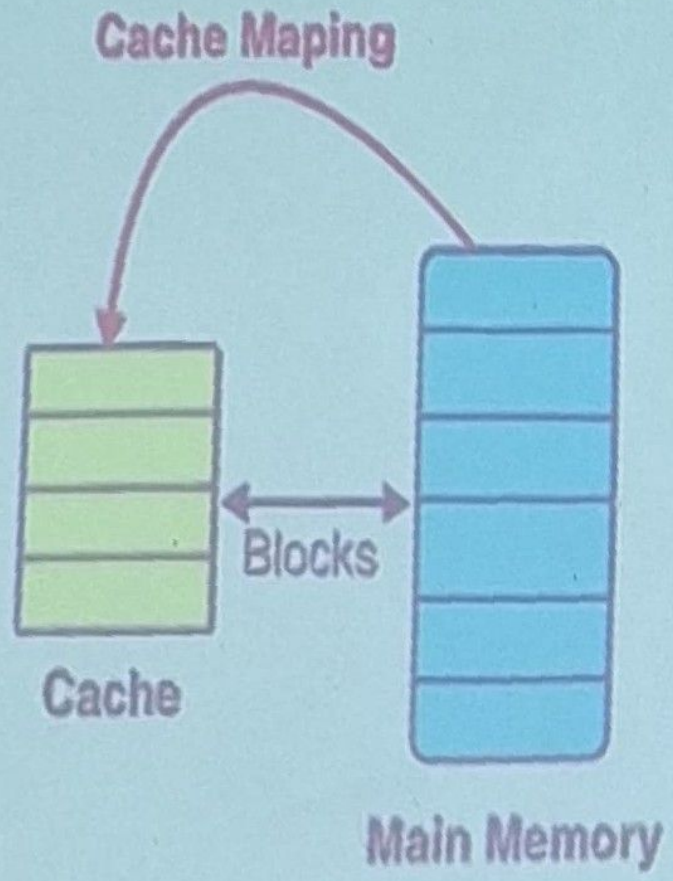


B is Block
L is Level

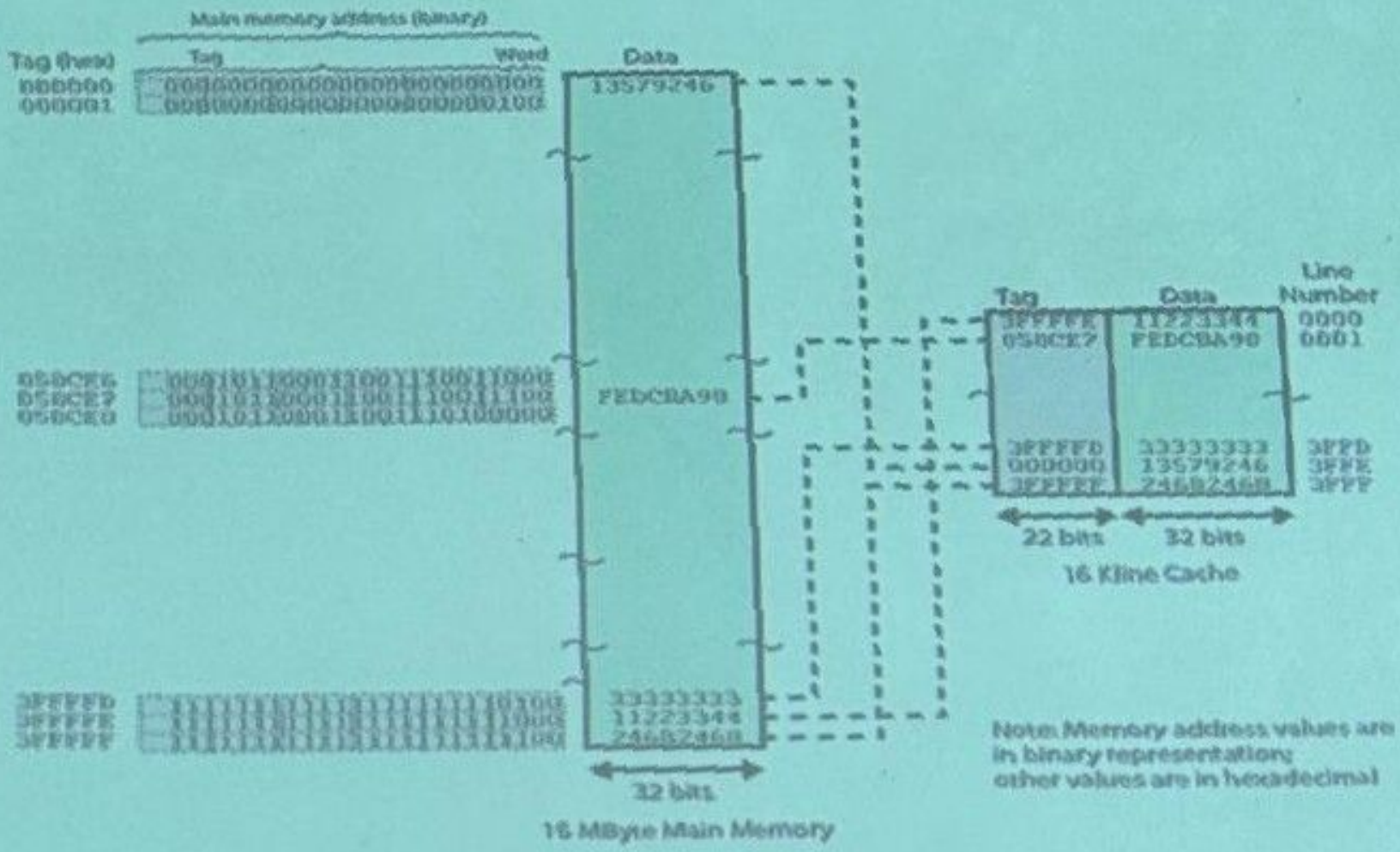


Cache/Memory Example

- Cache Size: 64 kByte
- Block size: 4 bytes.
- The cache has 16k (2^{14}) lines of 4 bytes
- Main memory: 16M bytes
- Address bus: 24-bit
- Blocks: 4M



Fully Associative Mapping
Address Structure



Associative Mapping Address Structure

Tag 22 bit	Word 2 bit

- 22 bit tag stored with each 32 bit block of data
- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which 16 bit word is required from 32 bit data block
- Example

Address	Tag	Data	Cache line
FFFFFC	FFFFFC	24682468	3FFF

Fully Associative Mapping Example 2

Assume that a portion of the tags in the cache in our example looks like the table below. Which of the following addresses are contained in the cache?

a.) $438EE8_{16}$ b.) $F18EFF_{16}$ c.) $6B8EF3_{16}$ d.) $AD8EF3_{16}$

Tag (binary)	Addresses w/ block			
	00	01	10	11
0101 0011 1000 1110 1110 10				
1110 1101 1100 1001 1011 01				
1010 1101 1000 1110 1111 00				
0110 1011 1000 1110 1111 11				
1011 0101 0101 1001 0010 00				
1111 0001 1000 1110 1111 11				

Fully Associative Mapping Example 3

Fully Associative Mapping Question (Byte Addressable Memmory)

Given Values FindValues

Physical Address

Tag	Block offset

Main Memory	Cache Size	Block Size	Tag Bits in PA	Tag Directory Size (Bit)	Comparator
128KB PA = 17bit	16KB	256B	9	$9 \cdot 2^5$	64
32GB PA = 35bit	32KB	1KB	25	$25 \cdot 2^5$	32
128MB PA = 27bit	512KB	1KB	17	$17 \cdot 2^9$	512
16GB PA = 34bit	4MB	4KB	22	$22 \cdot 2^{12}$	4,096

Work Addressable Memory

If word size is 8B (Divide Main Memory, Cache Memory and Block Size by Word Size)

Main Memory	Cache Size	Block Size	Tag Bits in PA	Tag Directory Size (Bit)	Comparator
128KB/8 = 16KW PA = 14bit	16KB/8 = 2KW	256B/8 = 32W = 5-bit	9	$9 \cdot 2^5$	64

Fully Associative Mapping Example 4

A certain processor uses a fully associative cache of size 16 kB, The cache block size is 16 bytes. Assume that the main memory is byte addressable and uses a 32-bit address. How many bits are required for the Tag and the Index fields respectively in the addresses generated by the processor?

32- bit

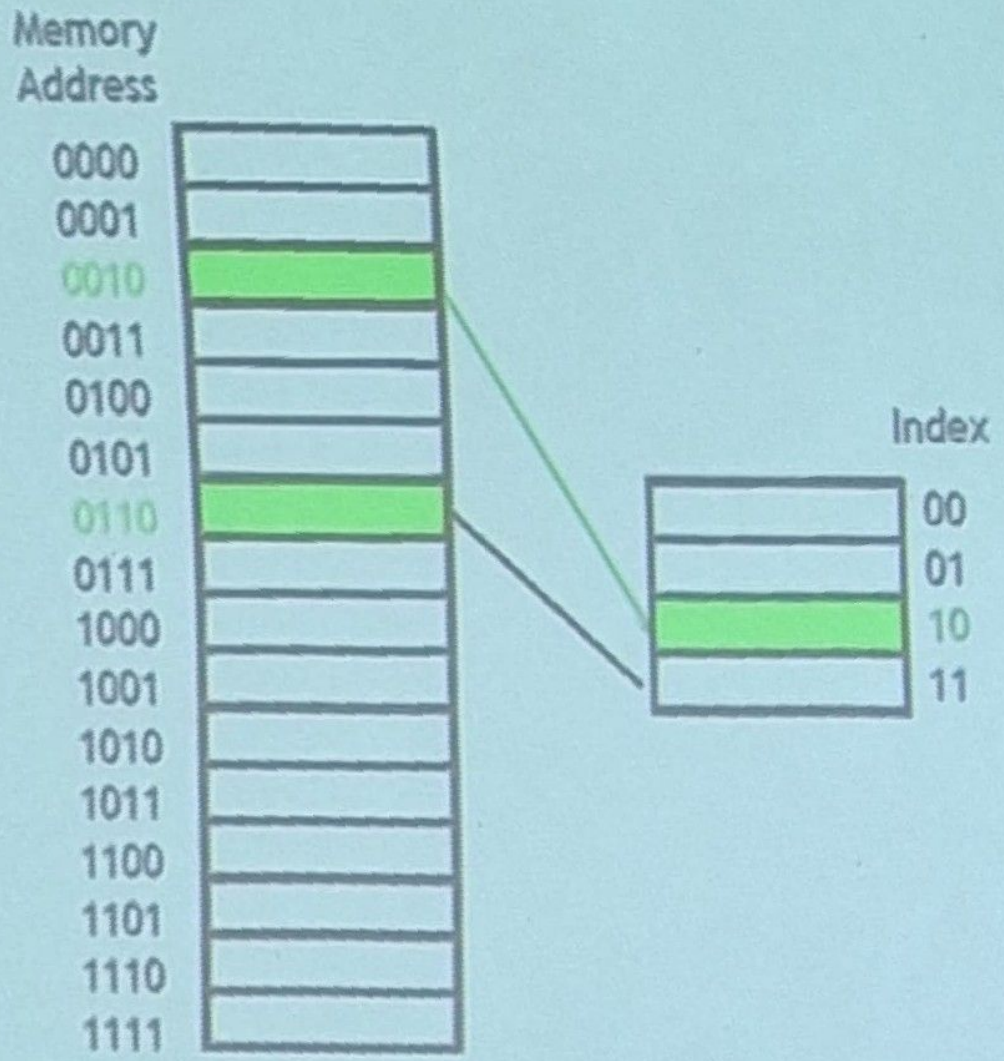
Tag offset	Word Offset
28 bit	4 bit

Fully Associative Mapping Advantages

- A fully associative cache permits data to be stored in any cache block, instead of forcing each memory address into one particular block.
  - When data is fetched from memory, it can be placed in any unused block of the cache.
  - This way we'll never have a conflict between two or more memory addresses which map to a single cache block.
- We might put memory address 2 in cache block 2, and address 6 in block 3.
- Then subsequent repeated accesses to 2 and 6 would all be hits instead of misses.

Fully Associative Mapping Advantages

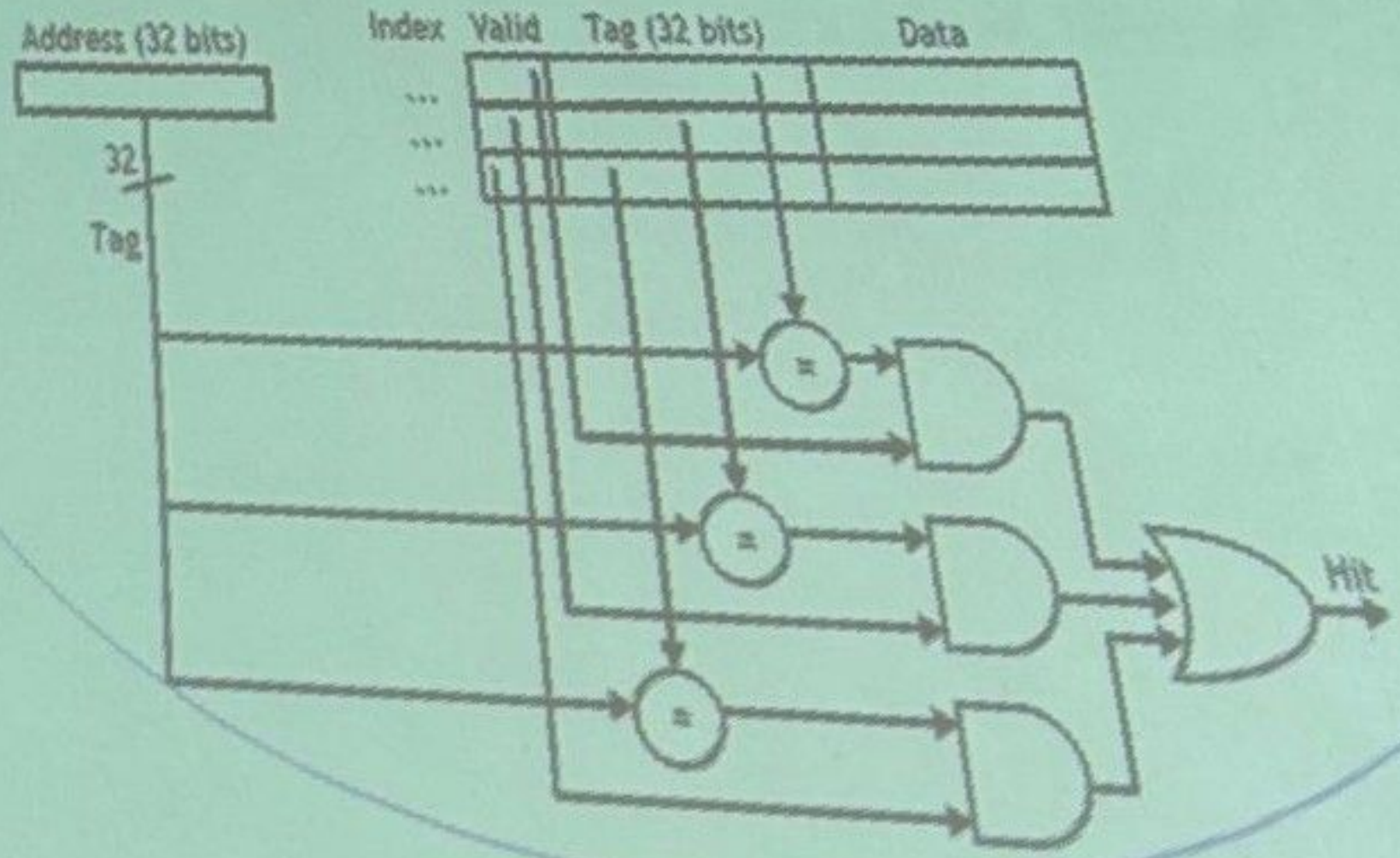
- If all the blocks are already in use, it's usually best to replace the *least recently used* one, assuming that if it hasn't used it in a while, it won't be needed again anytime soon.



Fully Associative Mapping Limitations

- *A fully associative cache is expensive to implement.*
  - Because there is no index field in the address anymore, the entire address must be used as the tag, increasing the total cache size.
- *Data could be anywhere in the cache, so we must check the tag of every cache block.*
  - That's a lot of comparators!
- *Every line's tag must be examined for a match*
  - The mapping is slower

Fully Associative Mapping Limitations



Set Associative

Cache Mapping

K-way set associative